Guide de comprØhension - TaskFlow

Objectif : comprendre l'application de bout en bout et pouvoir dØfendre ses choix techniques pendant un entretien.

TaskFlow est une application web de gestion de projets et de tâches. Elle possŁde une interface React, une API FastAPI et une base de donnØes MySQL.

Conseil : ne mØmorise pas ce document mot àmot. Ouvre les fichiers indiquØs, exØcute les commandes, puis explique-les avec tes propres mots.

1. Vue d'ensemble de l'architecture

Navigateur -> frontend React -> Nginx -> API FastAPI -> SQLAlchemy -> MySQL.

Le frontend affiche les pages et recueille les actions de l'utilisateur. Le backend dØcide si une action est autorisØe, valide les donnØes et les enregistre. MySQL conserve les donnØes mŒme aprŁs le redØmarrage des conteneurs.

Dans Docker Compose, les services sont frontend, backend et db. Le frontend est accessible sur http://localhost:8080, Swagger sur http://localhost:8001/docs et le contrôle de santØ sur http://localhost:8001/health.

Le rØseau interne isole MySQL : son port n'est pas publiØ vers le poste hôte. Les autres conteneurs le joignent avec le nom db.

2. Exemple complet : crØer une tâche

1. L'utilisateur se connecte puis sØlectionne un projet dans Dashboard.jsx.

2. React utilise api() dans frontend/src/api.js pour envoyer une requŒte POST JSON à /api/v1/projects/{id}/tasks.

3. Le jeton JWT est ajoutØ dans l'en-tŒte Authorization: Bearer <token>.

4. FastAPI dirige la requŒte vers backend/app/api/routes/tasks.py.

5. deps.py vØrifie le JWT, identifie l'utilisateur courant et ouvre une session de base de donnØes.

6. Le backend vØrifie que le projet appartient bien àcet utilisateur. Sinon il rØpond 404.

7. Pydantic valide les champs : titre, statut, prioritØ et date Øventuelle.

8. SQLAlchemy Øcrit la tâche dans MySQL, puis FastAPI rØpond 201 Created avec du JSON.

9. React recharge les donnØes et affiche la nouvelle tâche.

3. Backend FastAPI : organisation

backend/app/main.py crØe l'application FastAPI, configure CORS, les gestionnaires d'erreurs, /health et les routeurs.

backend/app/api/routes/auth.py contient l'inscription et la connexion. projects.py gŁre les projets. tasks.py gŁre les tâches, les filtres, la recherche et les statistiques.

backend/app/api/deps.py contient les dØpendances rØutilisables : session de base de donnØes et utilisateur courant. C'est important car cela Øvite de recopier le contrôle de sØcuritØ dans chaque route.

backend/app/models contient les modŁles SQLAlchemy. User possŁde plusieurs Project ; Project possŁde plusieurs Task.

backend/app/schemas contient les modŁles Pydantic utilisØs pour valider les entrØes et dØfinir les rØponses JSON. Un schØma API n'est pas obligatoirement identique àune table de base de donnØes.

backend/app/core/config.py lit les variables d'environnement. backend/app/core/security.py gŁre le hash des mots de passe et les JWT.

backend/alembic contient les migrations. Au dØmarrage, alembic upgrade head applique le schØma de base de donnØes attendu.

4. Authentification et autorisation

Authentification : vØrifier qui est l'utilisateur. Lors de la connexion, le mot de passe fourni est comparØ àson hash BCrypt stockØ. Si la vØrification rØussit, l'API produit un JWT signØ.

Autorisation : vØrifier ce que cet utilisateur peut faire. Avant de lire, modifier ou supprimer un projet ou une tâche, le backend filtre avec l'identifiant du propriØtaire du projet.

Phrase d'entretien : l'authentification prouve l'identitØ ; l'autorisation vØrifie les droits sur une ressource.

Pourquoi rØpondre 404 pour une ressource d'un autre utilisateur ? Cela Øvite aussi de confirmer qu'une ressource appartenant àquelqu'un d'autre existe.

5. Frontend React et Vite

React construit l'interface avec des composants. App.jsx dØfinit les routes Login, Register et Dashboard. ProtectedRoute redirige vers /login si aucun jeton n'est prØsent.

auth.jsx stocke l'Øtat de connexion et le jeton. api.js centralise fetch : URL API, en-tŒtes JSON, jeton Bearer et gestion uniforme des erreurs.

Dashboard.jsx contient les Øtats React pour les projets, les tâches, les formulaires, les filtres, le chargement et les erreurs. components.jsx regroupe les composants visuels rØutilisables.

Vite n'est pas React. Vite est l'outil qui lance rapidement le serveur de dØveloppement et produit les fichiers statiques optimisØs pour la production.

6. DØfendre le choix FastAPI

RØponse courte : j'ai choisi FastAPI pour construire une API REST moderne, lisible et rapidement testable. Il s'intŁgre àPydantic pour la validation, àSQLAlchemy pour MySQL, et gØnŁre automatiquement OpenAPI et Swagger.

Avantages concrets dans ce projet : validation typØe, dØpendances pour l'authentification et la base de donnØes, rØponses HTTP claires, documentation interactive à/docs, tests simples avec pytest.

Limite honnŒte : sur un systŁme beaucoup plus grand, j'ajouterais par exemple la limitation de dØbit, du cache, une observabilitØ plus complŁte, des tâches asynchrones et une sØparation plus poussØe des domaines mØtier.

7. DØfendre le choix React + Vite

RØponse courte : TaskFlow est une interface interactive avec formulaires, filtres, statut de connexion, chargement et donnØes qui Øvoluent sans recharger toute la page. React est adaptØ àcette gestion d'Øtat par composants.

Vite rØduit la configuration et fournit un dØmarrage rapide en dØveloppement ainsi qu'un build statique optimisØ. En production, Nginx sert ce build, ce qui est lØger et fiable.

Limite honnŒte : React introduit davantage de logique côtØ client qu'une page HTML rendue côtØ serveur. Ici ce compromis est justifiØ par l'expØrience de tableau de bord dynamique.

8. Docker, CI/CD et qualitØ

docker-compose.yml orchestre les trois services. Les health checks permettent àDocker de savoir si MySQL, le backend et le frontend sont prŒts. Le volume mysql_data garde les donnØes entre les redØmarrages.

Le backend utilise un Dockerfile Python et lance les migrations avant Uvicorn. Le frontend utilise un build Node puis Nginx pour servir les fichiers produits.

Les tests backend couvrent les unitØs, l'API, l'authentification, l'autorisation et les entrØes invalides. Le frontend possŁde au moins un test de connexion. Le smoke test vØrifie le parcours local aprŁs Docker.

GitHub Actions rØpŁte lint, tests et build sur les pull requests. L'objectif est d'empŒcher la publication d'une version si les vØrifications Øchouent.

9. Questions frØquentes pour la soutenance

Pourquoi un ORM ? SQLAlchemy rend les modŁles lisibles et produit des requŒtes paramØtrØes, ce qui aide àprØvenir l'injection SQL.

Pourquoi Alembic ? Les migrations versionnent le schØma. Un autre environnement peut recrØer la mŒme base sans crØer des tables manuellement.

Pourquoi MySQL n'est pas exposØ ? La base est privØe dans le rØseau Docker interne ; cela rØduit la surface d'attaque.

Que contient le JWT ? Une identitØ signØe et une expiration. Le frontend le transmet ; le backend le valide avant une route protØgØe.

Comment revenir en arriŁre aprŁs une mauvaise livraison ? Revenir àune image versionnØe connue ou àun tag Git, puis redØployer l'image correspondante. Le plan dØtaillØ est dans CLOUD_DEPLOYMENT.md.

10. Plan personnel de rØvision

1. Ouvre main.py, deps.py, security.py, projects.py et tasks.py. Explique leur rôe sans regarder ce guide.

2. CrØe deux comptes. Avec Swagger, vØrifie qu'un compte ne peut pas accØder aux projets de l'autre.

3. Lance docker compose up -d --build, puis explique chaque service et chaque health check.

4. Lance pytest, ruff, npm run lint, npm run test et npm run build. Lis au moins un test backend et Login.test.jsx.

5. PrØpare une dØmonstration de 5 à10 minutes : inscription, projet, tâche, filtre, Swagger, persistance MySQL, tests, Docker, GitHub Actions et plan de dØploiement.

6. Sois transparent sur l'aide IA : explique ce que tu as vØrifiØ, adaptØ et compris. Ne prØtends jamais connaître un dØtail que tu ne peux pas expliquer.